

FIN 4600 Study Guide: Pretty Good Privacy (PGP)

Use this guide before and after the two lab sessions. It covers what you need to create a key, check someone else's key, and send and receive signed, encrypted files.

Learning objectives

After this lab you should be able to:

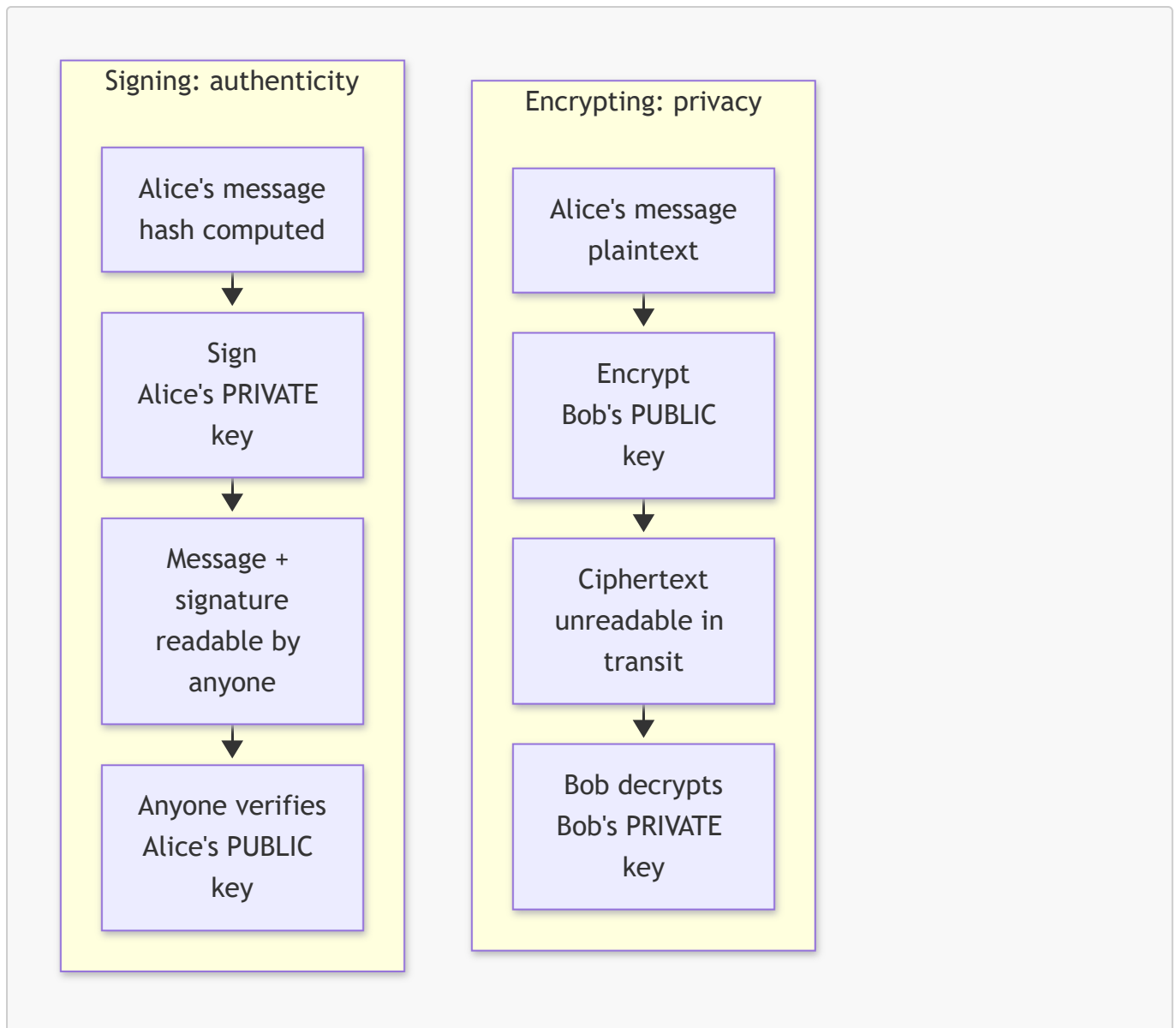
1. Explain the difference between encrypting and signing, and which key each one uses.
2. Describe the parts of an OpenPGP key: primary key, user ID, subkeys, self-signatures, and revocation certificate.
3. Explain what a self-signature proves and what it does not.
4. Verify a public key by comparing its fingerprint through a second channel, then certify it.
5. Explain what a keyserver is, and why creating a key does not publish it.
6. Connect PGP to real financial workflows such as encrypted file transfers between banks.

The one rule to remember

Encrypt with the recipient's public key. Sign with your own private key.

Encryption answers *who can read this?* Signing answers *who wrote this, and was it changed?* A signed message is not secret. An encrypted but unsigned message could have come from anyone.

Encrypting vs. signing



Step	Encrypting (privacy)	Signing (authenticity and integrity)
1	Alice writes a message (plaintext)	Alice writes a message; a hash of it is computed
2	Encrypt with Bob's public key	Sign the hash with Alice's private key
3	Ciphertext: unreadable in transit	Message plus signature: still readable by anyone
4	Bob decrypts with Bob's private key	Anyone verifies with Alice's public key

Each flow uses one public key and one private key, in opposite order.

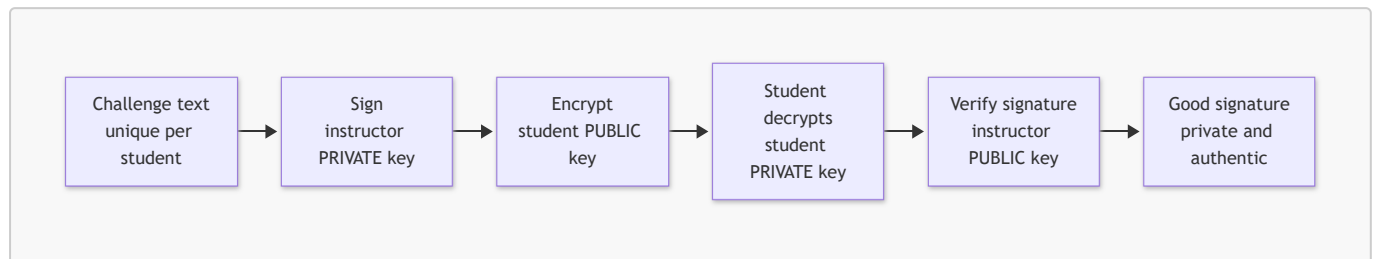
Hybrid encryption

PGP never encrypts a whole file with RSA or ECC, because public-key math is slow. It creates a random one-time **session key**, encrypts the file with a fast symmetric cipher (AES), and then encrypts only the session key with the recipient's public key. To encrypt to several people, PGP wraps the same session key once for each recipient.

What a digital signature is

GnuPG hashes the message (for example with SHA-256) and signs the hash with the sender's private key. Changing even one character changes the hash, so verification fails. In the lab, changing \$5,000 to \$50,000 in a signed payment instruction makes the signature check fail.

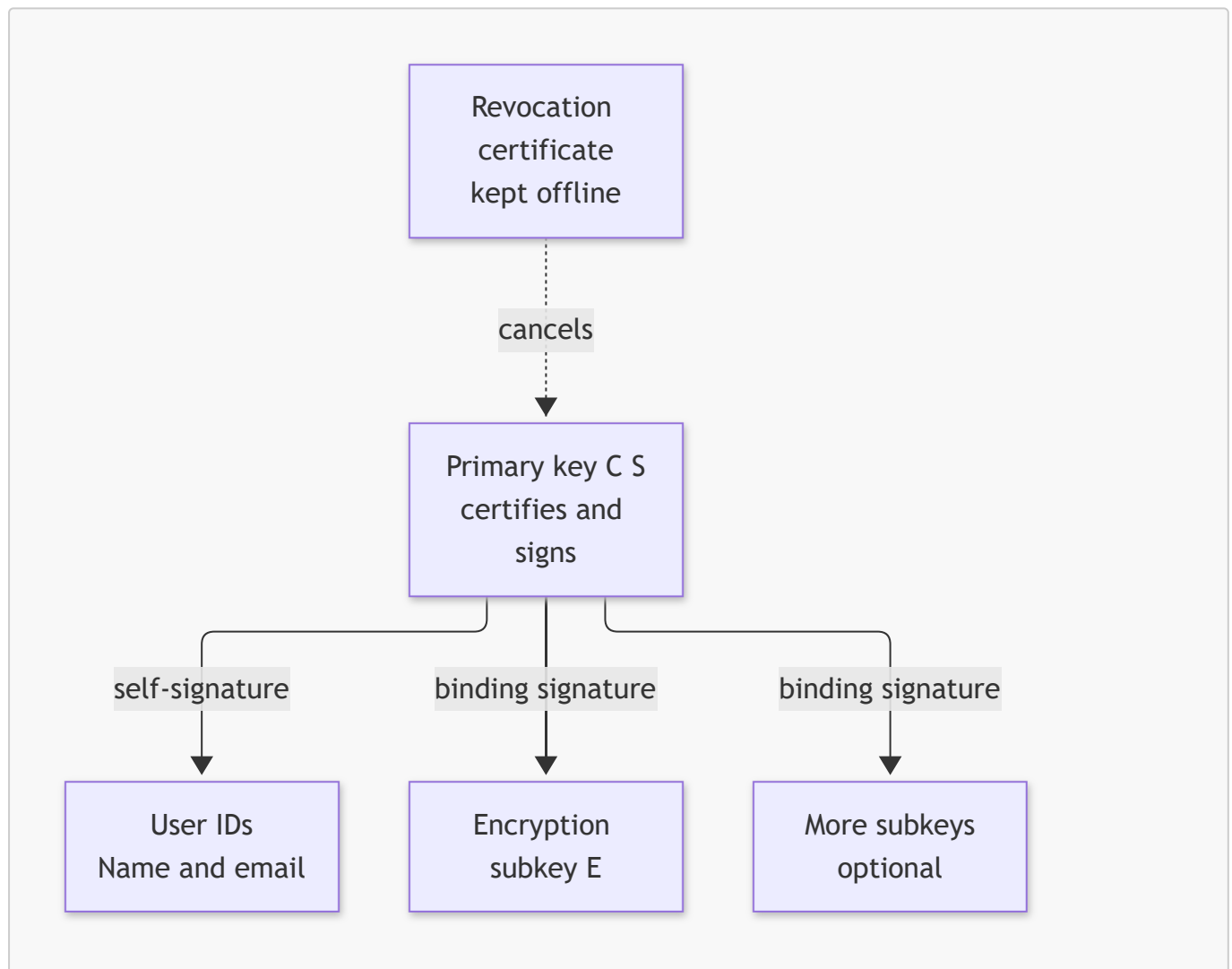
Your challenge file: sign, then encrypt



Your reply reverses the roles: you sign with your private key and encrypt to the instructor's public key.

Anatomy of your key

Clicking **New Key Pair** creates a bundle, not a single key. One fingerprint identifies the whole bundle.



Part	What it does
Primary key <i>CS</i>	Certifies the other parts and signs. Its fingerprint names the whole key.
User ID	Your name and email. A key can have more than one.
Self-signature	The primary key's signature on each user ID and subkey, tying the bundle together.
Encryption subkey <i>E</i>	The key people actually encrypt to.
Revocation certificate	Pre-signed notice that cancels the key if it is lost or stolen. GnuPG saves one in <code>openpgp-revocs.d</code> . Back it up.

What a self-signature proves: whoever holds the private key attached this name. **What it does not prove:** that the name is true. Anyone can create a key claiming to be your instructor.

Trust: fingerprints and certification

A key file proves nothing by itself. You make it trustworthy by comparing its **fingerprint** against one you received through a **different channel**, such as the board, a phone call, or in person. Then you **certify** it (*Certify* in Kleopatra, *lsign* in GnuPG). An attacker who swaps the key file on Canvas would also have to change the fingerprint on the board. Using two channels defeats a man-in-the-middle attack.

PGP uses a **web of trust**: any user can vouch for any key. HTTPS uses **certificate authorities**: a small set of organizations that browsers trust.

Where keys live

Creating a key is purely local. It stays in your keyring until you choose to upload it. A **keyserver** is an optional public directory.

- **keys.openpgp.org** publishes your name and email only after you click a verification link, and lets you remove them.
- The older **SKS** network could not delete keys. It was abused in 2019 and shut down in 2021.
- Companies often publish keys on their own domain with **Web Key Directory (WKD)**, or use commercial directories.
- **In this class we use Canvas, not a keyserver.** It keeps your email private and lets us practice fingerprint checking. Mac users: decline any upload offer from GPG Keychain.

PGP in finance

Banks, payment processors, and custodians exchange payroll files, ACH batches, and settlement reports as PGP-encrypted, signed files, usually over SFTP. When two firms start working together, they swap public keys and confirm fingerprints by phone or in a signed agreement. That is exactly the process you practice in this lab.

Key terms checklist

Check each term once you can explain it in your own words.

- ☐ **Plaintext / ciphertext**: readable data / encrypted, unreadable data
- ☐ **Public key**: shared freely; used to encrypt to you and to verify your signatures
- ☐ **Private (secret) key**: never shared; used to decrypt and to sign
- ☐ **Encryption**: makes data readable only by the holder of the matching private key
- ☐ **Digital signature**: proof of who signed, and that the content has not changed
- ☐ **Hash**: fixed-size digest of data; any change gives a different hash
- ☐ **Session key**: random one-time symmetric key for the bulk encryption
- ☐ **Hybrid encryption**: fast symmetric cipher for the data, public-key encryption for the session key
- ☐ **OpenPGP**: the open standard behind PGP (RFC 9580)
- ☐ **GnuPG (gpg)**: the free OpenPGP software used by Kleopatra and GPG Suite
- ☐ **Key pair / certificate**: your public and private key (Kleopatra calls it a *certificate*)
- ☐ **Primary key**: the central key that certifies the other parts of your key
- ☐ **Subkey**: an extra key bound to the primary key, often used for encryption
- ☐ **User ID**: the name and email attached to a key
- ☐ **Self-signature**: the primary key's signature on its own user IDs and subkeys
- ☐ **Fingerprint**: the unique identifier of a key (40 hex characters)
- ☐ **Certify / Isign**: sign someone else's key after checking its fingerprint (Isign keeps it local)
- ☐ **Web of trust**: trust built from users vouching for each other's keys
- ☐ **Certificate authority**: a central organization that vouches for keys, as used in HTTPS
- ☐ **Man-in-the-middle**: an attacker who substitutes their own key during key exchange
- ☐ **Revocation certificate**: a pre-made notice that cancels a lost or stolen key
- ☐ **Keyserver**: an optional public directory of public keys
- ☐ **Web Key Directory (WKD)**: keys published on the email domain's own website
- ☐ **Keyring**: your local store of your keys and others' public keys
- ☐ **Pinentry**: the window that asks for your passphrase

Self-check questions

1. You want to send your advisor a document only they can read. Whose key do you encrypt with: public or private?
2. You want to prove a document came from you. Which key do you use?
3. In the lab demo, why couldn't Alice decrypt the message she had just encrypted to Bob?
4. A signed message is intercepted. Can the interceptor read it? Can they change it without being detected?
5. Why does PGP use a session key instead of encrypting the whole file with RSA?
6. What does a self-signature prove? What does it not prove?
7. Why did we compare the instructor's fingerprint from the board instead of trusting the file on Canvas?
8. Does creating a key in Kleopatra publish it to the internet?
9. What should you do if your laptop with your private key is stolen?
10. Name one way banks use PGP today.

Answers

1. The advisor's **public** key.

2. Your own **private** key.
3. It was encrypted to Bob's public key, so only Bob's private key opens it, and Alice doesn't have it.
4. Yes, they can read it, because signing is not encryption. No, they can't change it: the signature check would fail.
5. Public-key math is slow. A fast symmetric cipher encrypts the data, and the public key protects only the small session key.
6. It proves the key holder attached the name. It does not prove the name is true.
7. An attacker could replace the file. A second channel means they would have to fake both.
8. No. It stays in your local keyring unless you upload it.
9. Publish your revocation certificate, create a new key, and tell your contacts.
10. Encrypted, signed payroll, ACH, or settlement files over SFTP; verifying signed software releases.

Lab checklist

Session 1

- ☐ Installed Gpg4win (Windows) or GPG Suite (macOS), and ran `pip install -r requirements.txt`
- ☐ Created **exactly one** key pair with your @mtu.edu address and a strong passphrase
- ☐ Ran Part A (Alice and Bob) and answered Questions 1 to 3
- ☐ Ran Part B and found your primary key, subkey, and self-signatures
- ☐ Ran Part C: fingerprint matched the board, and the instructor key was certified
- ☐ Ran Part D: uploaded `my_public_key.asc` to Canvas and typed your fingerprint into the Canvas quiz
- ☐ Backed up your revocation certificate

Session 2

- ☐ Decrypted your challenge and saw **GOOD signature** from the instructor
- ☐ Wrote your code word and answer, then created `response.asc`
- ☐ Uploaded `response.asc` to Canvas

Common mistakes

Mistake	What to do
Created two key pairs	Delete the extra one in Kleopatra, or set <code>MY_FPR_OVERRIDE</code> in the notebook
Installed the package <code>gnupg</code>	<code>pip uninstall gnupg</code> , then <code>pip install python-gnupg</code>
Nothing happens when a cell runs	The passphrase window may be hidden behind other windows
Uploaded or committed your private key	Tell your instructor right away, revoke the key, and create a new one
Ran the notebook in Google Colab	Run it on your own laptop so it uses your Kleopatra or GPG Keychain keyring

Videos and resources

- [Computerphile: Public Key Cryptography](#) (about 6 minutes)
- [Computerphile: What are Digital Signatures?](#) (about 10 minutes)
- [Computerphile: Key Exchange Problems](#) (about 10 minutes)
- [Kevin's Guides: PGP Encryption with Kleopatra](#)
- [Gpg4win documentation](#) and [GPG Suite](#)
- keys.openpgp.org: how the keyserver works